Parallel-in-time evaluation of MCMC sample paths

1 Parallel-in-time evaluation of MCMC sample paths

Suppose a sampler evolves by a randomized update of the form

$$x_{t+1} = F(x_t, \xi_{t+1}), \qquad \xi_{t+1} \overset{\text{i.i.d.}}{\sim} \mu.$$

The Sampling unit studies such updates through the laws they generate, but this handout fixes one realization of the random inputs and asks a different question: can the resulting trajectory be recovered faster than one step at a time on parallel hardware? The object of study is therefore not the chain law itself but the deterministic path in the product space of trajectories.

Long trajectories still matter for the same reason as in the main notes. To estimate $\mathbb{E}_\pi[h(X)]$ under a target distribution π, one averages a function of the states along a long run,

$$\hat{\mu}_T = \frac{1}{T} \sum_{t=1}^{T} h(x_t),$$

and increasing T typically reduces Monte Carlo error, up to the usual issues of burn-in and dependence. For Langevin-type samplers, freezing the randomness turns that long-run sampling problem into a deterministic trajectory problem. This handout therefore sits between two parts of the main notes: it builds on the Sampling unit's gradient-based MCMC material and on the Optimization unit's treatment of fixed points, conditioning, and Newton/Gauss–Newton local models, and it prepares for later augmented-state and bridge-based sampling viewpoints where whole trajectories become computational objects.

1.1 From sampling to deterministic trajectory equations

Once the random numbers are fixed, a length-T sample path is no longer random at all: it is the solution of a deterministic system in the full trajectory $x_{1:T} \in (\mathbb{R}^d)^T$. The natural object is therefore a trajectory-space nonlinear least-squares problem, with fixed-point and rootfinding formulations as equivalent descriptions of the same equations. [1]

Concretely, many samplers can be written in the reparameterized form

$$x_t = F(x_{t-1}, \xi_t), \qquad \xi_t \overset{\text{i.i.d.}}{\sim} \mu, \tag{1}$$

for some deterministic map F. Fix a horizon T and a realization $\xi_{1:T}$. Define deterministic transition maps $f_t(x) := F(x, \xi_t)$. Then the sequential sampler computes the rollout

$$x_t = f_t(x_{t-1}), \qquad t = 1, \dots, T. \tag{2}$$

[1] This fixed-randomness trajectory viewpoint appears broadly in work on parallelizing nonlinear recursions and sequential samplers; see, e.g., Zoltowski et al. (2025, §2), Shih et al. (2023, §3), Lim et al. (2024, §3), and Gonzalez et al. (2024, §1). For background on state space models and parallel Newton methods, see Gonzalez (2026, Ch. 2).

With $\xi_{1:T}$ fixed, trajectory computation becomes the nonlinear least-squares problem

$$\min_{x_{1:T}\in(\mathbb{R}^d)^T} \mathcal{L}(x_{1:T}) := \frac{1}{2}\sum_{t=1}^{T}\|x_t - f_t(x_{t-1})\|_2^2 \quad \text{(fixed noise, fixed } x_0\text{)}.$$

Its global minimum value is 0, the minimizer is exactly the sequential rollout, and under differentiability of the f_t the objective has a single stationary point. The reason is structural: each term links only (x_{t-1}, x_t), so the system is triangular in time. We prove this in Proposition 1.5; that fact is what lets us focus on *how* to solve the trajectory equations quickly.

For bookkeeping it is helpful to define the *trajectory map* $\mathcal{T} : (\mathbb{R}^d)^T \to (\mathbb{R}^d)^T$ by

$$(\mathcal{T}(x_{1:T}))_1 := f_1(x_0), \qquad (\mathcal{T}(x_{1:T}))_t := f_t(x_{t-1}), \quad t = 2, \dots, T.$$

Then the rollout equations (2) are exactly the fixed-point condition

$$x_{1:T} = \mathcal{T}(x_{1:T}).$$

Equivalently, with the residual $r(x_{1:T}) := x_{1:T} - \mathcal{T}(x_{1:T})$, the sequential sample path is the unique trajectory with $r(x_{1:T}) = 0$. (We make this rootfinding formulation and its Jacobian structure explicit in Section 1.2.)

With the randomness fixed, *Picard and Gauss–Newton are simply different iterative solvers for this same deterministic rootfinding / least-squares problem*: Picard is successive approximation (a "first-order" method), while Gauss–Newton uses Jacobians (a "second-order" method) to reduce iteration count. In principle one could plug in many other optimizers (gradient descent, quasi-Newton, etc.); we focus on these two because they make the parallel structure clearest and their tradeoffs mirror the familiar first-order versus second-order story.

This does not change the MCMC algorithm. The goal is simply to recover, up to numerical tolerance, the same sample path as the sequential implementation, but by iterating on the full trajectory in parallel.

Why can this be faster? At any current guess $x_{1:T}^{(k)}$, the "local" quantities $f_t(x_{t-1}^{(k)})$ (and, for Gauss–Newton, their Jacobians) can be computed in parallel over t. What remains is the coupling across time, and in the cases we focus on it reduces to a scan, i.e. a parallel prefix computation with only $\mathcal{O}(\log T)$ depth. For ULA, Picard updates become a scan-friendly cumulative sum, and Gauss–Newton steps reduce to a scan-based solve of a block lower-bidiagonal linear system.

This gives the usual parallel tradeoff: a sequential rollout costs depth T, while K trajectory iterations cost roughly $\mathcal{O}(K \log T)$ depth with $\mathcal{O}(T)$ work per iteration. When drift or Jacobian evaluations can be batched across time, that reduction in depth can translate into a real wall-clock gain on GPUs or TPUs.

To make the main idea as concrete as possible, we focus almost entirely on one sampler: the unadjusted Langevin algorithm (ULA). ULA is a clean starting point because, after we condition on its Gaussian noise, each step is a differentiable map, and the additive structure of the update makes scan operations especially transparent. The same viewpoint extends, with varying fidelity, to reparameterized Gibbs, SGLD, and Metropolis-corrected samplers.

Throughout, T denotes the chain length and d the state dimension. We use $t \in \{1, \dots, T\}$ for the chain index and $k \in \{0, 1, 2, \dots\}$ for iterations of a trajectory solver (Picard or Gauss–Newton).

We postpone the discussion of *when* the trajectory solve is easy or hard—and why the burn-in/away-from-stationarity part of the chain can be computationally special—to the end of the note; see Sections 1.7 and 1.9.

1.2 Rootfinding and least squares in trajectory space

Optimization and rootfinding are linked by stationarity: under mild regularity, a local minimizer must satisfy $\nabla f(x) = 0$, which is itself a rootfinding problem. Conversely, any rootfinding problem $r(x) = 0$ can be written as the minimization of $\frac{1}{2}\|r(x)\|_2^2$.

For fixed randomness, the rollout recursion (2) becomes a triangular nonlinear system in the full trajectory. We keep the least-squares objective $\mathcal{L}$ as the main lens, and we use the fixed-point and rootfinding views to expose the structure that matters for algorithms. Making the residual explicit will let us state the uniqueness/geometry facts and see the sparsity that makes scan-based parallelization possible.

Throughout this note we move freely between three equivalent viewpoints on the fixed-noise trajectory problem:

1. the least-squares problem $\min_{x_{1:T}} \mathcal{L}(x_{1:T}) = \frac{1}{2}\|r(x_{1:T})\|_2^2$;
2. the fixed-point equation $x_{1:T} = \mathcal{T}(x_{1:T})$ from Section 1.1;
3. the rootfinding problem $r(x_{1:T}) = 0$, where $r = x_{1:T} - \mathcal{T}(x_{1:T})$.

These are the standard optimization tools in different clothes: Picard is fixed-point iteration on the trajectory equations, gradient descent is fixed-point iteration on $x \mapsto x - \alpha\nabla\mathcal{L}(x)$, and Gauss–Newton/Newton uses Jacobians of r to accelerate convergence. When $\mathcal{L}(x) = \frac{1}{2}\|r(x)\|_2^2$, Gauss–Newton is just that idea applied to the trajectory residual; see Section 1.5.

Definition 1.1 (Trajectory residual)**.** Fix x_0 and the transition maps $f_{1:T}$. For a candidate trajectory $x_{1:T} = (x_1, \dots, x_T) \in (\mathbb{R}^d)^T$, define residual blocks

$$r_1(x_{1:T}) := x_1 - f_1(x_0), \qquad r_t(x_{1:T}) := x_t - f_t(x_{t-1}), \quad t = 2, \dots, T, \tag{3}$$

and let $r(x_{1:T}) \in (\mathbb{R}^d)^T$ denote the stacked vector $(r_1, \dots, r_T)$.

It is often helpful to interpret these residuals as *defects* from the rollout constraints. Define $\delta_t := r_t(x_{1:T})$. Then any defect sequence $\delta_{1:T}$ determines a unique trajectory via the forward recursion

$$x_1 = f_1(x_0) + \delta_1, \qquad x_t = f_t(x_{t-1}) + \delta_t, \quad t = 2, \dots, T,$$

and conversely, any trajectory determines its defects. Thus the map $x_{1:T} \mapsto \delta_{1:T}$ is a triangular change of variables, and in defect coordinates the objective is just

$$\mathcal{L}(x_{1:T}) = \frac{1}{2}\sum_{t=1}^{T}\|\delta_t\|_2^2.$$

This makes the uniqueness of the minimizer feel inevitable: the only way to drive $\mathcal{L}$ to zero is $\delta_t = 0$ for all t, i.e. the exact rollout. The Jacobian viewpoint below formalizes the same idea: the residual map has a block lower-triangular Jacobian with identity blocks on the diagonal, so $\nabla\mathcal{L} = J^\top r$ vanishes only when $r = 0$.

Notice that with the trajectory map $\mathcal{T}$ from Section 1.1, the stacked residual in Definition 1.1 is exactly

$$r(x_{1:T}) = x_{1:T} - \mathcal{T}(x_{1:T}).$$

We will use the corresponding merit function

$$\mathcal{L}(x_{1:T}) := \frac{1}{2}\|r(x_{1:T})\|_2^2,$$

which turns the fixed-noise trajectory problem into a deterministic nonlinear least-squares objective.

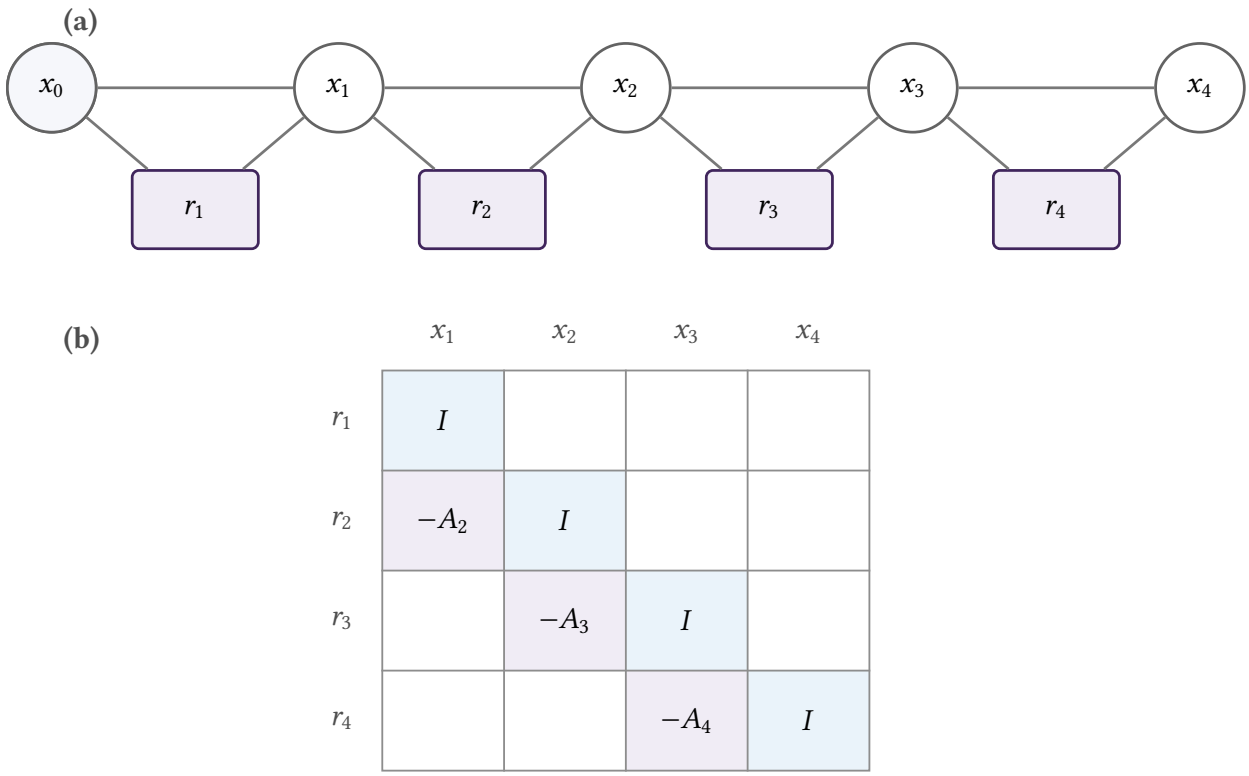

Figure 1: Left: each residual block $r_t(x_{t-1}, x_t) = x_t - f_t(x_{t-1})$ couples only adjacent states. Right: the Jacobian $J = \frac{\partial r}{\partial x_{1:T}}$ is block lower bidiagonal, with diagonal blocks I_d and subdiagonal blocks $-\frac{\partial f_t}{\partial x}(x_{t-1})$.

Proposition 1.2 (Correctness of the residual formulation). *A trajectory $x_{1:T}$ satisfies the rollout recursion (2) if and only if $r(x_{1:T}) = 0$.*

Proof. The statement is a direct restatement of (3). ■

The Jacobian of $r(x_{1:T})$ with respect to $x_{1:T}$ inherits the Markov structure: each block r_t depends only on (x_{t-1}, x_t). This sparsity is what makes Newton-like methods attractive here. See Figure 1 for a compact visual summary.

Assume each f_t is differentiable and write $A_t(x) := \frac{\partial f_t}{\partial x}(x) \in \mathbb{R}^{d\times d}$ for the single-step Jacobian.

Proposition 1.3 (Jacobian structure). *The Jacobian matrix $J(x_{1:T}) := \frac{\partial r}{\partial x_{1:T}}(x_{1:T}) \in \mathbb{R}^{Td\times Td}$ is block lower bidiagonal, with diagonal blocks I_d and subdiagonal blocks $-A_t(x_{t-1})$ for $t = 2, \dots, T$.*

Proof. Differentiate the residual blocks r_t from (3). For $t = 1$, $r_1(x_{1:T}) = x_1 - f_1(x_0)$, so $\frac{\partial r_1}{\partial x_1} = I_d$ and $\frac{\partial r_1}{\partial x_j} = 0$ for $j \neq 1$. For $t \geq 2$, $r_t(x_{1:T}) = x_t - f_t(x_{t-1})$, so

$$\frac{\partial r_t}{\partial x_t} = I_d, \qquad \frac{\partial r_t}{\partial x_{t-1}} = -A_t(x_{t-1}), \qquad \frac{\partial r_t}{\partial x_j} = 0 \text{ for } j \notin \{t-1, t\}.$$

Stacking these block derivatives yields the claimed block lower bidiagonal structure. ■

Corollary 1.4 (The Gauss–Newton linear solve is always well-defined)**.** *For any candidate trajectory $x_{1:T}$, the Jacobian $J(x_{1:T})$ is invertible. Moreover, solving $J(x_{1:T})\,\Delta x_{1:T} = -r(x_{1:T})$ is equivalent to solving a forward recursion in t.*

Proof. By Proposition 1.3, $J(x_{1:T})$ is block lower triangular with diagonal blocks equal to I_d. Therefore $J(x_{1:T})$ is invertible (its determinant is 1). The forward-recursion claim is the usual forward substitution for a lower bidiagonal linear system. ■

Proposition 1.5 (Key geometry fact (triangular least squares))**.** *Fix x_0 and deterministic transition maps $f_{1:T}$, and assume each f_t is differentiable. Define the trajectory residual $r(x_{1:T})$ by (3) and the merit function $\mathcal{L}(x_{1:T}) = \frac{1}{2}\|r(x_{1:T})\|_2^2$. Then:*

1. *The rollout recursion $x_t = f_t(x_{t-1})$ has a unique solution $x^\star_{1:T}$. Equivalently, $\mathcal{L}$ has a unique global minimizer $x^\star_{1:T}$ and $\mathcal{L}(x^\star_{1:T}) = 0$.*
2. *$\mathcal{L}$ has a unique stationary point, namely $x^\star_{1:T}$. In particular, $\mathcal{L}$ has no spurious local minima.*

Proof. 1. The recursion $x_1 = f_1(x_0),\ x_2 = f_2(x_1), \dots,\ x_T = f_T(x_{T-1})$ uniquely determines $(x_1, \dots, x_T)$; call the result $x^\star_{1:T}$. By construction $r(x^\star_{1:T}) = 0$, hence $\mathcal{L}(x^\star_{1:T}) = 0$. Since $\mathcal{L}(x_{1:T}) = \frac{1}{2}\|r(x_{1:T})\|_2^2 \geq 0$ for all $x_{1:T}$, $x^\star_{1:T}$ is a global minimizer. If $\mathcal{L}(\hat{x}_{1:T}) = 0$, then $r(\hat{x}_{1:T}) = 0$, so by Proposition 1.2 the trajectory $\hat{x}_{1:T}$ satisfies the rollout recursion and must equal $x^\star_{1:T}$.

2. By the chain rule,
$$\nabla\mathcal{L}(x_{1:T}) = J(x_{1:T})^\top r(x_{1:T}),$$
where $J(x_{1:T}) = \frac{\partial r}{\partial x_{1:T}}(x_{1:T})$. By Corollary 1.4, $J(x_{1:T})$ is invertible for every $x_{1:T}$, hence so is $J(x_{1:T})^\top$. Therefore $\nabla\mathcal{L}(x_{1:T}) = 0$ if and only if $r(x_{1:T}) = 0$, which (by the first part) holds only at $x^\star_{1:T}$.

■

Proposition 1.5 is the key geometry fact: for fixed randomness, the trajectory objective $\mathcal{L}$ has a single stationary point and no spurious local minima, even though it is generally nonconvex. The question is therefore not *which* trajectory the solver might find, but *whether* a particular iterative method converges quickly and stably. This is the same kind of "basin of attraction / conditioning" issue that appears throughout optimization: the problem has a unique solution, but an algorithm can still be fragile or slow depending on the geometry it encounters along the way; see Section 1.7. That is the main contrast with the nonconvex examples in the Optimization unit and with the expectation–maximization subsection in the Augmented-state optimization unit: here the fixed-noise objective may be nonconvex, but it does not create bad local minima, so the hard part is conditioning and basin size of the iterative solver rather than the existence of competing optima.

1.3 ULA warm-up: quadratic potential ⇒ an affine recursion

We begin with the simplest ULA setting, where the potential is quadratic and the target is Gaussian. In that case each step is affine, so the full trajectory can be recovered from a single prefix computation.

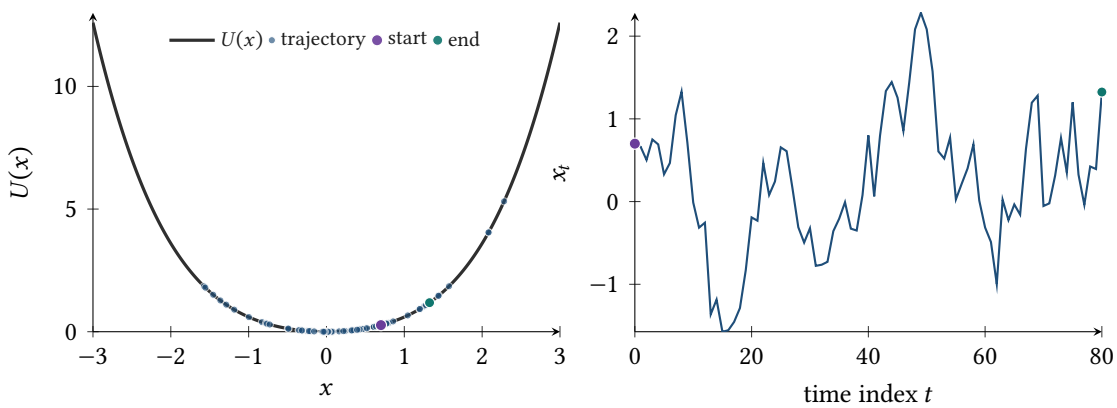

Figure 2: A one-dimensional energy U and a sequential ULA trajectory for one fixed noise realization. *Left:* the potential curve with the visited states drawn in the $(x, U(x))$ plane. *Right:* the same trajectory as a time series.

1.3.1 Unadjusted Langevin algorithm (ULA)

Suppose the target density has the form $\pi(x) \propto e^{-U(x)}$ on $\mathbb{R}^d$. In the Sampling unit's gradient-based MCMC subsection, the ULA recursion appears as

$$x_{t+1} = x_t - \epsilon \nabla U(x_t) + \sqrt{2\epsilon}\, \xi_{t+1}.$$

Here we keep the full path indexed as $x_0, \dots, x_T$, so each trajectory constraint naturally couples (x_{t-1}, x_t). Accordingly, we relabel the same update by one step and write

$$x_t = x_{t-1} - \epsilon \nabla U(x_{t-1}) + \sqrt{2\epsilon}\, \xi_t, \qquad \xi_t \sim \mathcal{N}(0, I_d). \tag{4}$$

This is only a bookkeeping convention for the trajectory solve; the algorithm itself is unchanged. The only randomness is the Gaussian noise ξ_t; conditional on ξ_t, the update is deterministic.

Definition 1.6 (ULA rollout with fixed noise)**.** Fix an initial state x_0 and realized noise vectors $\xi_1, \dots, \xi_T$. Define the per-step update maps

$$f_t(x) := x - \epsilon \nabla U(x) + \sqrt{2\epsilon}\, \xi_t, \qquad t = 1, \dots, T.$$

The sequential ULA implementation is the recursion

$$x_t = f_t(x_{t-1}), \qquad t = 1, \dots, T, \tag{5}$$

which uniquely determines the sample path $x_{1:T} = (x_1, \dots, x_T)$ once the noise is fixed.

Example 1.7 (Quadratic potential $\Rightarrow$ affine recursion)**.** Let $U(x) = \frac{1}{2} x^\top Q x$ with $Q \succeq 0$. Then $\nabla U(x) = Qx$, and (4) becomes

$$x_t = A x_{t-1} + b_t, \qquad A := I_d - \epsilon Q, \qquad b_t := \sqrt{2\epsilon}\, \xi_t. \tag{6}$$

For fixed randomness $(\xi_t)_{t=1}^T$, evaluating the MCMC trajectory is exactly the task of evaluating an affine recursion.

So the "parallel-in-time" question becomes whether the whole sequence $(x_t)_{t=1}^T$ can be computed from the per-step affine maps without stepping one time index at a time.

1.3.2 Composing affine updates in parallel

Consider a general time-varying affine recursion

$$x_t = A_t x_{t-1} + b_t, \qquad t = 1, \dots, T, \tag{7}$$

where $A_t \in \mathbb{R}^{d\times d}$ and $b_t \in \mathbb{R}^d$ are given. Define the per-step affine maps $\varphi_t : \mathbb{R}^d \to \mathbb{R}^d$ by $\varphi_t(z) = A_t z + b_t$.

Lemma 1.8 (Composing affine maps)**.** *Let $\varphi(z) = Az + b$ and $\psi(z) = Cz + d$ be affine maps on $\mathbb{R}^d$. Then $\psi \circ \varphi$ is affine, with*

$$(\psi \circ \varphi)(z) = (CA)z + (Cb + d).$$

In particular, affine maps are closed under composition, and composition is associative.

Proof. For any $z \in \mathbb{R}^d$,

$$(\psi \circ \varphi)(z) = \psi(Az + b) = C(Az + b) + d = (CA)z + (Cb + d),$$

which is again of the form "matrix times z plus vector". Associativity follows because function composition is associative. ■

Proposition 1.9 (Prefix compositions solve the affine recursion)**.** *Define the prefix compositions*

$$\Phi_t := \varphi_t \circ \varphi_{t-1} \circ \cdots \circ \varphi_1, \qquad t = 1, \dots, T.$$

We use Φ_t for these affine prefix compositions (distinct from the trajectory map $\mathcal{T}$ in Section 1.1). Then the recursion (7) satisfies $x_t = \Phi_t(x_0)$ for all t. Moreover, each Φ_t is affine: there exist $P_t \in \mathbb{R}^{d\times d}$ and $q_t \in \mathbb{R}^d$ such that $\Phi_t(z) = P_t z + q_t$. These coefficients satisfy

$$P_1 = A_1, \quad q_1 = b_1, \qquad P_t = A_t P_{t-1}, \quad q_t = A_t q_{t-1} + b_t, \qquad t = 2, \dots, T.$$

In particular, $x_t = P_t x_0 + q_t$ for all t.

Proof. The recursion says $x_t = \varphi_t(x_{t-1})$, so unrolling gives $x_t = \Phi_t(x_0)$.

For the affine form, note that $\Phi_1 = \varphi_1$, so $P_1 = A_1$ and $q_1 = b_1$. Assume $\Phi_{t-1}(z) = P_{t-1}z + q_{t-1}$. Then

$$\Phi_t(z) = \varphi_t(\Phi_{t-1}(z)) = A_t(P_{t-1}z + q_{t-1}) + b_t = (A_t P_{t-1})z + (A_t q_{t-1} + b_t),$$

so $P_t = A_t P_{t-1}$ and $q_t = A_t q_{t-1} + b_t$. Evaluating Φ_t at $z = x_0$ gives $x_t = P_t x_0 + q_t$. ■

At this point the sequential dependence of (7) has been pushed into one object, the prefix compositions $\Phi_t = \varphi_t \circ \cdots \circ \varphi_1$. Proposition 1.9 says that once these are known, every state is just $x_t = \Phi_t(x_0)$. The remaining question is whether all prefix compositions can be computed without a length-T sequential loop.

To see where the logarithm comes from, it helps to start with a very concrete case: cumulative sums on $\mathbb{R}$. Given numbers $s_1, \dots, s_T$, define

$$p_t := s_t + s_{t-1} + \cdots + s_1.$$

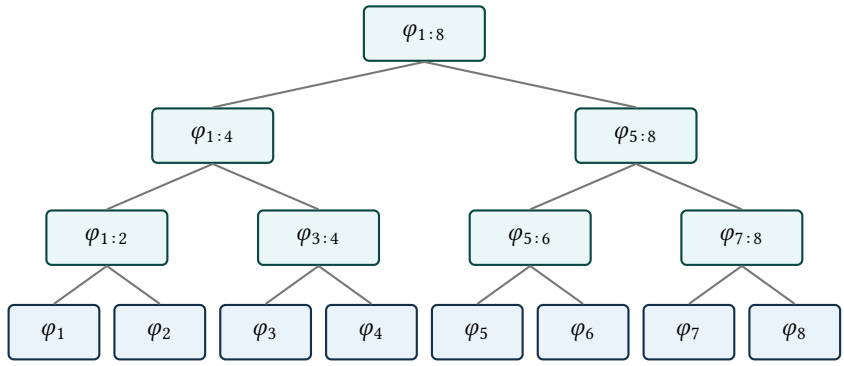

Figure 3: Divide-and-conquer tree of compositions (the up-sweep) for affine maps. A scan uses this kind of balanced tree to compute all prefix compositions in only $\mathcal{O}(\log T)$ parallel rounds (rather than T sequential rounds).

The sequential recursion $p_t = s_t + p_{t-1}$ is a length-T loop.

Now suppose for simplicity that $T = 2^m$. In one parallel round we can form the pairwise sums

$$u_j := s_{2j} + s_{2j-1}, \qquad j = 1, \dots, T/2.$$

By associativity, the even prefixes satisfy

$$p_{2j} = u_j + u_{j-1} + \cdots + u_1,$$

so computing all even prefixes reduces to computing prefix sums for a list of length $T/2$. Repeating the same pairing step halves the length each round, so after $m = \log_2 T$ rounds the list has length one. To obtain all prefixes (including the odd indices), one keeps these partial sums in the balanced tree created by the repeated pairings and performs a short reverse pass that distributes the correct left-prefix to each subtree; Figure 3 shows this tree.

Nothing in this argument used anything special about addition beyond associativity. The same idea applies to *function composition*, which is also associative and is exactly what we need for (7). Concretely, we take the balanced tree in Figure 3 and replace each "+" with "$\circ$": each parallel round composes many disjoint pairs at once, and a short reverse pass distributes the correct left-prefix to each subtree.

Definition 1.10 (Scan / prefix compositions)**.** Given maps $g_1, \dots, g_T : \mathbb{R}^d \to \mathbb{R}^d$, the (inclusive) **scan** returns the sequence of prefix compositions

$$G_t := g_t \circ g_{t-1} \circ \cdots \circ g_1, \qquad t = 1, \dots, T.$$

The order in Definition 1.10 is chosen so that G_t means "apply step 1, then 2, and so on up to t."The same pairing argument shows that scans can be implemented in $\mathcal{O}(\log T)$ parallel rounds with $\mathcal{O}(T)$ total work, because the only property we used was associativity. This is the recurring pattern in the later algorithms: once the across-time coupling is rewritten as an affine recursion, scan does the rest.

In our affine-recursion setting, the scan inputs are the affine maps $\varphi_t(z) = A_t z + b_t$. The scan outputs are the prefix compositions $\Phi_t = \varphi_t \circ \cdots \circ \varphi_1$. By Proposition 1.9, each Φ_t has the form $z \mapsto P_t z + q_t$, so once we have computed all (P_t, q_t) we can recover the whole trajectory via $x_t = P_t x_0 + q_t$. Figure 3 shows the basic divide-and-conquer tree underlying this computation.

The next exercise is the scalar version of the same affine-prefix identity.

Exercise 1.11 (A scalar warm-up: expanding an affine recursion)**.** Consider the one-dimensional recursion $x_t = ax_{t-1} + b_t$ with $a \in \mathbb{R}$ fixed. Show that

$$x_t = a^t x_0 + \sum_{s=1}^{t} a^{t-s} b_s.$$

Explain in one sentence why this is compatible with the scan viewpoint above.

Solution

This is the scalar unrolling that the scan packages in parallel. We expand by induction. For $t = 1$, $x_1 = ax_0 + b_1$, which matches the formula. Assume $x_{t-1} = a^{t-1}x_0 + \sum_{s=1}^{t-1} a^{t-1-s} b_s$. Then

$$x_t = ax_{t-1} + b_t = a\left(a^{t-1}x_0 + \sum_{s=1}^{t-1} a^{t-1-s} b_s\right) + b_t = a^t x_0 + \sum_{s=1}^{t-1} a^{t-s} b_s + b_t = a^t x_0 + \sum_{s=1}^{t} a^{t-s} b_s.$$

The scan viewpoint packages these same expansions using associativity: instead of forming powers a^{t-s} explicitly, it composes the per-step affine maps in a divide-and-conquer way to obtain all prefixes in $\mathcal{O}(\log T)$ depth.

The quadratic case is special because the rollout recursion is affine, so one scan produces the entire trajectory. For a general energy U, the maps f_t in Definition 1.6 are nonlinear, so there is no single scan that recovers $x_{1:T}$ from x_0.

A natural next step is to ask whether we can still reuse the scan idea *inside an iterative solver*. We start with the simplest such solver: Picard iteration (successive approximation), which only uses evaluations of ∇U.

1.4 Picard iteration: parallel fixed-point updates for ULA

The Picard part of the story uses only fixed-point iteration on the trajectory equations, so the per-iteration work is just drift or increment evaluation together with a scan. Its convergence is linear, which is why contraction, damping, or windowing matter.

1.4.1 ULA in cumulative-sum form

Fix a noise realization $\xi_{1:T}$. For notational convenience, define the *increment* map

$$g_t(x) := -\epsilon \nabla U(x) + \sqrt{2\epsilon}\, \xi_t, \qquad t = 1, \dots, T.$$

Then the ULA recursion (4) can be written as

$$x_t = x_{t-1} + g_t(x_{t-1}), \qquad t = 1, \dots, T. \tag{8}$$

Summing (8) from 1 to t gives the equivalent cumulative form

$$x_t = x_0 + \sum_{s=1}^{t} g_s(x_{s-1}), \qquad t = 1, \dots, T. \tag{9}$$

Viewed as an equation in the unknown path $x_{1:T}$, (9) is a fixed-point formulation of the rollout. It has the same solution as the step-by-step recursion (5), but it makes the scan structure explicit. There are

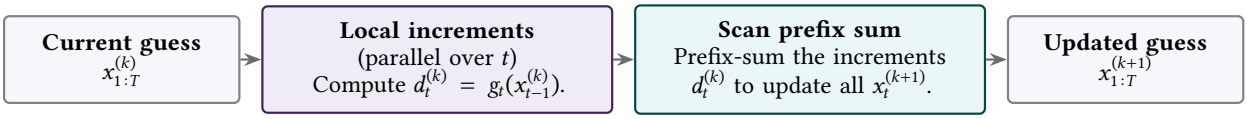

Figure 4: One Picard iteration on the full trajectory: compute the increments $g_t(x_{t-1}^{(k)})$ in parallel across time, then use a scan (prefix sum) to integrate those increments and update the whole path.

multiple equivalent fixed-point formulations. For example, the direct Picard map $x_t^{(k+1)} = f_t(x_{t-1}^{(k)})$ computes all f_t in parallel with $\mathcal{O}(1)$ depth, while the cumulative-sum form (9) exposes a scan. We focus on the scan-friendly form because it matches the same "evaluate many time indices, then combine them" pattern that appears in modern bridge-based diffusion samplers, and it aligns with the scan-based linear solve used by Gauss–Newton.

Equation (9) suggests a simple parallel strategy: if we had a good guess of the states $(x_{t-1})_{t=1}^T$, we could evaluate all increments $g_t(x_{t-1})$ in parallel, and then recover $(x_t)_{t=1}^T$ by a single prefix sum. This leads to Picard iteration.

1.4.2 Picard iteration and where scan enters

Starting from a guess $x_{1:T}^{(0)}$, define iterates $x_{1:T}^{(k)}$ by

$$x_t^{(k+1)} = x_0 + \sum_{s=1}^{t} g_s\left(x_{s-1}^{(k)}\right), \qquad t = 1, \dots, T, \tag{10}$$

where we interpret $x_0^{(k)} = x_0$ for all k. If the iteration converges, its limit must satisfy (9), hence is the same trajectory as the sequential rollout (5).

Proposition 1.12 (One Picard update is parallel increments + a scan)**.** *Fix a Picard iterate $x_{1:T}^{(k)}$ and define the per-time increments*

$$d_t^{(k)} := g_t\left(x_{t-1}^{(k)}\right) = -\epsilon \nabla U(x_{t-1}^{(k)}) + \sqrt{2\epsilon}\, \xi_t, \qquad t = 1, \dots, T.$$

Then the next iterate satisfies

$$x_t^{(k+1)} = x_0 + \sum_{s=1}^{t} d_s^{(k)}, \qquad t = 1, \dots, T.$$

In particular, once the $d_t^{(k)}$ are available (parallel over t), forming $x_{1:T}^{(k+1)}$ is exactly a prefix-sum scan.

Proof. This is a direct restatement of (10). ■

In practice one often adds *damping* for robustness: given the undamped update $\tilde{x}_{1:T}^{(k+1)}$ from (10), set

$$x_{1:T}^{(k+1)} = (1-\omega) x_{1:T}^{(k)} + \omega \tilde{x}_{1:T}^{(k+1)}, \qquad \omega \in (0, 1].$$

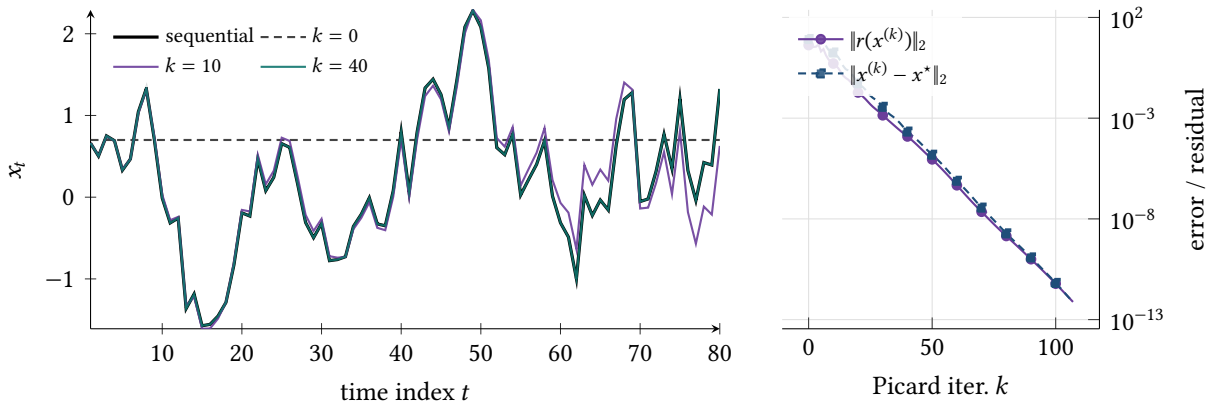

Figure 5: Empirical behavior of damped Picard iteration on a one-dimensional ULA example with fixed noise (same example as Figure 7). Picard uses only increment evaluations and scans, so it is a clean parallel baseline, but it converges linearly and can require many iterations without a good contraction regime.

Algorithm 1 Parallel-in-time Picard iteration for ULA with fixed noise (damped)

Input: Initial state x_0, energy U (with ∇U), step size ϵ, noise $\xi_{1:T}$, initial guess $x^{(0)}_{1:T}$, damping $\omega \in (0, 1]$, tolerance τ

Output: Approximate ULA trajectory $x_{1:T}$

for $k = 0, 1, 2, \ldots$ until convergence **do**
  For $t = 1, \ldots, T$ compute $d^{(k)}_t \leftarrow -\epsilon \nabla U(x^{(k)}_{t-1}) + \sqrt{2\epsilon}\, \xi_t$ ▷ all t in parallel
  Compute $s^{(k)}_t \leftarrow \sum_{i=1}^{t} d^{(k)}_i$ for all t ▷ via scan / prefix sum
  Set $\tilde{x}^{(k+1)}_t \leftarrow x_0 + s^{(k)}_t$ for all t
  Update $x^{(k+1)}_{1:T} \leftarrow (1 - \omega) x^{(k)}_{1:T} + \omega \tilde{x}^{(k+1)}_{1:T}$
  if $\max_{t \in \{1,\ldots,T\}} \|x^{(k+1)}_t - x^{(k)}_t\|_2 \le \tau$ **then**
    break
  end if
end for
return $x^{(k+1)}_{1:T}$

1.4.3 When does Picard converge? why windows help

The drawback of (10) is that it needs the fixed-point map in (9) to be stable. A simple bound explains both why global Picard can be hard for long chains and why sliding windows are a natural practical fix.

Proposition 1.13 (A simple contraction bound for Picard on a horizon). *Assume ∇U is L-Lipschitz on a set containing all states that appear in the Picard iterates. Consider the undamped Picard map $\mathcal{P}$ defined by (10), i.e. $x^{(k+1)} = \mathcal{P}(x^{(k)})$. Then for any two candidate trajectories $x_{1:T}$ and $y_{1:T}$,*

$$\max_{t \in \{1,\ldots,T\}} \|\mathcal{P}(x)_t - \mathcal{P}(y)_t\|_2 \le \epsilon L T \max_{t \in \{0,\ldots,T-1\}} \|x_t - y_t\|_2,$$

where $x_0 = y_0 = x_0$ is fixed. In particular, if $\epsilon L T < 1$ then $\mathcal{P}$ is a contraction (in the max norm) and Picard iteration converges. More generally, the same bound with T replaced by a window length *p shows that Picard is much easier to make contracting on short windows.*

Proof. Fix $t \in \{1, \dots, T\}$. The noise terms cancel, so

$$\mathcal{P}(x)_t - \mathcal{P}(y)_t = -\epsilon \sum_{s=1}^{t} \big(\nabla U(x_{s-1}) - \nabla U(y_{s-1})\big).$$

Taking norms and using Lipschitzness gives

$$\|\mathcal{P}(x)_t - \mathcal{P}(y)_t\|_2 \le \epsilon \sum_{s=1}^{t} \|\nabla U(x_{s-1}) - \nabla U(y_{s-1})\|_2 \le \epsilon L \sum_{s=1}^{t} \|x_{s-1} - y_{s-1}\|_2 \le \epsilon L t \max_{j \in \{0,\dots,T-1\}} \|x_j - y_j\|_2.$$

Taking the maximum over $t \le T$ yields the claim. ■

Exercise 1.14 (Window length and Picard stability)**.** Assume ∇U is L-Lipschitz on a set containing all states of interest. Fix a left boundary value x_t and consider applying Picard iteration only on the next p steps, i.e. to solve (9) for $(x_{t+1}, \dots, x_{t+p})$ while holding x_t fixed.

1. Show that the corresponding Picard map on this length-p window has Lipschitz constant at most $\epsilon L p$ in the max norm.
2. Explain briefly why this suggests a *sliding-window Picard* method for long trajectories, and how scan would be used inside each window.

Solution

1. The windowed Picard map is the same as in Proposition 1.13, but the sums run only over the next p steps. Repeating the bound in the proof of Proposition 1.13 with $t \le p$ gives

$$\max_{j \in \{1,\dots,p\}} \|\mathcal{P}_{\text{win}}(x)_{t+j} - \mathcal{P}_{\text{win}}(y)_{t+j}\|_2 \le \epsilon L p \max_{j \in \{0,\dots,p-1\}} \|x_{t+j} - y_{t+j}\|_2,$$

   where $x_t = y_t$ is fixed.

2. If $\epsilon L T$ is not small, global Picard may fail to contract over the whole horizon, but the same dynamics can be contracting over shorter horizons. A sliding-window method tries to exploit this by repeatedly running Picard updates on a window of length p and then shifting the window forward once the early part has stabilized. Inside each window, one Picard update still consists of (i) evaluating increments $g_s(x_{s-1})$ in parallel over the window indices, and (ii) integrating them via a prefix sum, i.e. a scan.

1.4.4 Sliding-window Picard

The window-length bound in Proposition 1.13 suggests a practical way to make Picard useful even when T is large: pick a window length p, run one or a few Picard updates on the current window, and slide the window forward once the early part has stabilized. Each update still has the same parallel structure as in Figure 4: drift evaluations in parallel across the window indices, followed by a scan to form the cumulative sums.

This sliding-window idea is especially natural when the expensive step is evaluating a learned vector field across many time indices. In many bridge-based diffusion samplers, batching those per-time evaluations over a short window can yield substantial wall-clock speedups even if the total number of model evaluations increases. That is why closely related windowed schemes appear there as well; see Shih et al. (2023, §3.1).

Picard and Gauss–Newton are two ends of the same story: they are iterative solvers for the same fixed-noise trajectory problem (equivalently, minimizing $\mathcal{L} = \frac{1}{2}\|r(x_{1:T})\|_2^2$). Picard uses only function evaluations (increments) plus scans, but it can require many iterations unless the dynamics are strongly contracting. Gauss–Newton uses Jacobians to accelerate convergence when the map is not a contraction. We now turn to Gauss–Newton updates on trajectories.

1.5 Gauss–Newton updates on trajectories

Gauss–Newton uses the same trajectory objective, but each iteration now linearizes the residual and minimizes the local least-squares model for $\mathcal{L}$. For ULA this means evaluating drifts and Jacobians, which are Hessians of U, and then solving the induced block lower-bidiagonal system with a scan. The method is most effective when the transitions are close to affine along the path, and damping is the usual safeguard when they are not.

We now apply a second-order method to the trajectory objective

$$\mathcal{L}(x_{1:T}) = \tfrac{1}{2}\|r(x_{1:T})\|_2^2.$$

Since $\mathcal{L}$ is a nonlinear least-squares objective, a standard choice is the Gauss–Newton method, which minimizes the squared norm of the *linearized* residual; see Exercise 1.19. In our Markovian setting this leads to scan-friendly linear algebra across time.

In our trajectory residual formulation, the map $r : (\mathbb{R}^d)^T \to (\mathbb{R}^d)^T$ is *square*, and its Jacobian is invertible for every trajectory (Corollary 1.4). As a result, the Gauss–Newton step for minimizing $\mathcal{L}$ is equivalent to Newton's method for the rootfinding problem $r(x_{1:T}) = 0$ (see Exercise 1.19). We keep the Gauss–Newton language because we think of the method as a least-squares solver, and because the same perspective extends to settings where one uses approximate Jacobians or over/underdetermined residuals.

This is a (structured) nonlinear least-squares problem in $\mathbb{R}^{Td}$. It is *not* convex in general: even if U is convex, the squared rollout residual typically introduces nonconvexity because the residual depends nonlinearly on the previous state x_{t-1}. Convexity holds in the special affine case (equivalently, a quadratic U), where the rollout equations are linear in the full trajectory and $\mathcal{L}$ becomes a strongly convex quadratic. So, the "ease" of the parallel solve should not be thought of as a convexity guarantee. Instead, what matters is stability/conditioning of the fixed-noise dynamics along the path and whether the transitions are close to affine on the region the trajectory explores; see Sections 1.7 and 1.9.

This "merit function in trajectory space" is the objective that measures how far a candidate trajectory is from satisfying the rollout equations (3). Although $\mathcal{L}$ lives in $\mathbb{R}^{Td}$, the meaning of the iterates can be very concrete: each Gauss–Newton step produces an *entire candidate sample path* $x_{1:T}^{(k)}$. To keep the visualization uncluttered, we will illustrate this behavior in one dimension; Figure 7 shows the sequential rollout and the first few Gauss–Newton iterates for one fixed noise realization.

We will need the single-step Jacobians $A_t(x) = \frac{\partial f_t}{\partial x}(x)$. For ULA, this Jacobian has a simple closed form. The next exercise is the same one-line differentiation that underlies Newton's method in the Optimization unit, now applied to a sampler step.

Exercise 1.15 (Jacobian for ULA)**.** Assume U is twice differentiable. Show that for the ULA update maps f_t from Definition 1.6,

$$A_t(x) = \frac{\partial f_t}{\partial x}(x) = I_d - \epsilon\nabla^2 U(x).$$

Solution

For fixed ξ_t, the update map is

$$f_t(x) = x - \epsilon \nabla U(x) + \sqrt{2\epsilon}\, \xi_t.$$

The derivative of $x \mapsto x$ is I_d. The derivative of $x \mapsto -\epsilon \nabla U(x)$ is $-\epsilon \nabla^2 U(x)$. The noise term is constant in x, so its derivative is 0. Therefore $\frac{\partial f_t}{\partial x}(x) = I_d - \epsilon \nabla^2 U(x)$.

Theorem 1.16 (Gauss–Newton step for the trajectory objective)**.** *Assume each f_t is differentiable. Given a current guess $x_{1:T}^{(k)}$, define*

$$A_t^{(k)} := A_t\left(x_{t-1}^{(k)}\right) \in \mathbb{R}^{d\times d}, \qquad r_t^{(k)} := r_t(x_{1:T}^{(k)}) \in \mathbb{R}^d.$$

Let $\Delta x_{1:T}^{(k)} = (\Delta x_1^{(k)}, \dots, \Delta x_T^{(k)})$ be the Gauss–Newton increment, i.e. the minimizer of the linearized least-squares model

$$\min_{\Delta x_{1:T} \in (\mathbb{R}^d)^T} \tfrac{1}{2} \| r(x_{1:T}^{(k)}) + J(x_{1:T}^{(k)})\, \Delta x_{1:T} \|_2^2.$$

Equivalently (since $J(x_{1:T}^{(k)})$ is always invertible by Corollary 1.4), $\Delta x_{1:T}^{(k)}$ is the unique solution of

$$J(x_{1:T}^{(k)})\, \Delta x_{1:T}^{(k)} = -r(x_{1:T}^{(k)}).$$

Then the blocks $\Delta x_t^{(k)} \in \mathbb{R}^d$ satisfy the linear recursion

$$\Delta x_1^{(k)} = -r_1^{(k)}, \qquad \Delta x_t^{(k)} = A_t^{(k)} \Delta x_{t-1}^{(k)} - r_t^{(k)}, \quad t = 2, \dots, T. \tag{11}$$

Moreover, the trajectory update $x_{1:T}^{(k+1)} = x_{1:T}^{(k)} + \Delta x_{1:T}^{(k)}$ can be written equivalently as

$$x_t^{(k+1)} = f_t\left(x_{t-1}^{(k)}\right) + A_t^{(k)} \left(x_{t-1}^{(k+1)} - x_{t-1}^{(k)}\right), \quad t = 1, \dots, T, \tag{12}$$

where we interpret $x_0^{(k+1)} = x_0^{(k)} = x_0$.

Proof of Theorem 1.16. By Proposition 1.3, $J(x_{1:T}^{(k)})$ is block lower bidiagonal with diagonal blocks I_d and subdiagonal blocks $-A_t^{(k)}$. Writing the block system $J\Delta x = -r$ coordinate-wise gives

$$\Delta x_1^{(k)} = -r_1^{(k)}, \qquad \Delta x_t^{(k)} - A_t^{(k)} \Delta x_{t-1}^{(k)} = -r_t^{(k)}, \quad t = 2, \dots, T,$$

which rearranges to (11).

Next, note that $\Delta x_t^{(k)} = x_t^{(k+1)} - x_t^{(k)}$ and $r_t^{(k)} = x_t^{(k)} - f_t(x_{t-1}^{(k)})$. For $t \geq 2$, substitute these identities into (11) to get

$$x_t^{(k+1)} - x_t^{(k)} = A_t^{(k)}\left(x_{t-1}^{(k+1)} - x_{t-1}^{(k)}\right) - \left(x_t^{(k)} - f_t(x_{t-1}^{(k)})\right),$$

and rearrange to obtain (12). For $t = 1$, the same formula holds when we interpret $x_0^{(k+1)} = x_0^{(k)} = x_0$. ■

Now the scan reappears in a very concrete way. Define $\Delta x_0^{(k)} := 0$ and $A_1^{(k)} := 0$. Then (11) can be written uniformly as

$$\Delta x_t^{(k)} = A_t^{(k)}\, \Delta x_{t-1}^{(k)} - r_t^{(k)}, \qquad t = 1, \dots, T, \tag{13}$$

which is an affine recursion of the form (7). In other words, *the Gauss–Newton linear solve is itself a rollout problem*, but now for an affine system.

Proposition 1.17 (The Gauss–Newton linear solve is a scan over affine maps)**.** *Fix an iteration k and define affine maps $\psi_t^{(k)} : \mathbb{R}^d \to \mathbb{R}^d$ by*

$$\psi_t^{(k)}(v) := A_t^{(k)} v - r_t^{(k)}, \qquad t = 1, \dots, T,$$

with $\Delta x_0^{(k)} := 0$. Then the Gauss–Newton increment satisfies

$$\Delta x_t^{(k)} = (\psi_t^{(k)} \circ \psi_{t-1}^{(k)} \circ \cdots \circ \psi_1^{(k)})(0), \qquad t = 1, \dots, T.$$

In particular, computing all $\Delta x_t^{(k)}$ is exactly a scan (prefix composition) over the affine maps $\psi_t^{(k)}$.

Proof. The recursion (13) is exactly $\Delta x_t^{(k)} = \psi_t^{(k)}(\Delta x_{t-1}^{(k)})$ with $\Delta x_0^{(k)} = 0$. Unrolling the recursion gives the claimed prefix-composition formula. ■

Proposition 1.17 is the clean point where scan enters the parallel-in-time Gauss–Newton method: once the local pieces $A_t^{(k)}$ and $r_t^{(k)}$ are available (parallel over t), the remaining "sequential" part of the step is a scan. Equivalently, the linear solve is forward substitution for a block lower-bidiagonal system; a scan is a parallel way to carry out that forward substitution.

Proposition 1.18 (Affine transitions are solved in one Gauss–Newton step)**.** *Assume each transition is affine: $f_t(x) = A_t x + b_t$. Then $r(x_{1:T})$ is an affine function of $x_{1:T}$, and one Gauss–Newton step finds the exact rollout from any initial guess $x_{1:T}^{(0)}$.*

Proof. If each f_t is affine, then each residual block $r_t(x_{1:T}) = x_t - f_t(x_{t-1})$ is affine in (x_{t-1}, x_t), hence r is affine in $x_{1:T}$. Therefore the Jacobian $J(x_{1:T})$ is constant in $x_{1:T}$ and the linearization is exact:

$$r(x_{1:T}^{(0)} + \Delta x) = r(x_{1:T}^{(0)}) + J\,\Delta x.$$

The Gauss–Newton step chooses Δx to minimize $\|r(x_{1:T}^{(0)}) + J\Delta x\|_2^2$. Since J is invertible (Corollary 1.4), this minimum is achieved at $r(x_{1:T}^{(0)}) + J\Delta x = 0$. By exact linearization, this gives $r(x_{1:T}^{(1)}) = 0$, which is the exact rollout by Proposition 1.2. ■

The next exercise reconnects this scan-based update to the standard nonlinear least-squares derivation from the optimization side of the course.

Exercise 1.19 (Gauss–Newton viewpoint)**.** Consider the merit function $\mathcal{L}(x_{1:T}) = \frac{1}{2}\|r(x_{1:T})\|_2^2$.

1. Compute $\nabla\mathcal{L}(x_{1:T})$ in terms of $J(x_{1:T})$ and $r(x_{1:T})$.
2. Show that a Gauss–Newton step for minimizing $\mathcal{L}$ solves $(J^\top J)\Delta x = -J^\top r$.
3. Use Corollary 1.4 to show this is equivalent to $J\Delta x = -r$ (i.e. the linearized residual can be driven to zero).
4. Give one reason you might prefer a damped or line-searched step on $\mathcal{L}$ rather than an undamped Gauss–Newton step, especially when the fixed-noise dynamics are expanding or poorly conditioned.

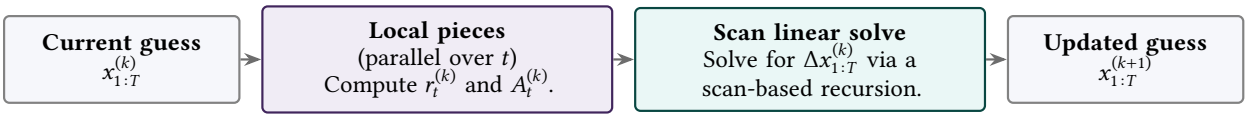

Figure 6: One Gauss–Newton iteration on the full trajectory: local residual/Jacobian evaluations parallel across time, followed by a scan-based solve of the induced linear recursion and an (optionally damped) trajectory update.

Solution

1. Let $r = r(x_{1:T})$ and $J = J(x_{1:T})$. Using the chain rule for $\mathcal{L}(x) = \frac{1}{2}\langle r(x), r(x)\rangle$ gives
$$\nabla\mathcal{L}(x_{1:T}) = J(x_{1:T})^\top r(x_{1:T}).$$
2. A Gauss–Newton step minimizes the linearized least-squares model
$$\min_{\Delta x} \ \tfrac{1}{2}\|r(x_{1:T}) + J(x_{1:T})\,\Delta x\|_2^2.$$
Differentiating with respect to Δx gives the normal equations
$$J(x_{1:T})^\top\big(r(x_{1:T}) + J(x_{1:T})\,\Delta x\big) = 0,$$
i.e. $(J^\top J)\Delta x = -J^\top r$.
3. By Corollary 1.4, J is invertible, hence so is $J^\top$. Left-multiplying $(J^\top J)\Delta x = -J^\top r$ by $J^{-\top}$ yields $J\Delta x = -r$. Equivalently, the linearized residual $r + J\Delta x$ can be made exactly zero in one Gauss–Newton step.
4. Undamped Gauss–Newton steps can diverge far from a solution (the linear model for r may be inaccurate). Minimizing $\mathcal{L}$ with damping/backtracking is often more robust because it enforces descent in $\mathcal{L}$, providing a safeguard when the local linearization is poor.

1.6 Putting it together: parallel Gauss–Newton rollout for ULA

We can now combine the pieces. Each Gauss–Newton iteration requires:

1. computing $f_t(x_{t-1}^{(k)})$, $A_t^{(k)}$, and $r_t^{(k)}$ for all t (parallel across t), and
2. solving the affine recursion (11) for $\Delta x_t^{(k)}$ (parallel across t via scan).

Figure 6 shows this pipeline schematically.

Algorithm 2 Parallel-in-time Gauss–Newton rollout for ULA with fixed noise

Input: Initial state x_0, energy U (with ∇U and $\nabla^2 U$), step size ϵ, noise $\xi_{1:T}$, initial guess $x_{1:T}^{(0)}$, tolerance ε
Output: Approximate ULA trajectory $x_{1:T}$

```
for k = 0, 1, 2, ... until convergence do
    Compute r_1^(k) ← x_1^(k) − (x_0 − ε∇U(x_0) + √(2ε) ξ_1)
    For t = 2, ..., T compute r_t^(k) ← x_t^(k) − (x_{t−1}^(k) − ε∇U(x_{t−1}^(k)) + √(2ε) ξ_t)      ▷ all t in parallel
    For t = 2, ..., T compute A_t^(k) ← I_d − ε∇²U(x_{t−1}^(k))                                    ▷ all t in parallel
    Solve Δx_{1:T}^(k) from (11)                                                                   ▷ via parallel scan
    Update x_{1:T}^(k+1) ← x_{1:T}^(k) + Δx_{1:T}^(k)                                              ▷ optionally damped
    if ‖r(x_{1:T}^(k+1))‖_2 ≤ ε then
        break
    end if
end for
return x_{1:T}^(k+1)
```

As with ordinary Newton-type methods, one often adds *damping* for robustness: instead of taking the full step $x_{1:T}^{(k+1)} = x_{1:T}^{(k)} + \Delta x_{1:T}^{(k)}$, use

$$x_{1:T}^{(k+1)} = x_{1:T}^{(k)} + \eta^{(k)} \Delta x_{1:T}^{(k)}, \qquad \eta^{(k)} \in (0, 1],$$

with $\eta^{(k)}$ chosen (for example) by backtracking line search on the merit function $\mathcal{L}(x_{1:T}) = \frac{1}{2}\|r(x_{1:T})\|_2^2$. This is the same damping/backtracking idea used for Newton updates in the Optimization unit.

Proposition 1.20 (Why convergence implies correctness)**.** *If Algorithm 2 terminates at a trajectory $\hat{x}_{1:T}$ with $r(\hat{x}_{1:T}) = 0$, then $\hat{x}_{1:T}$ is exactly the sequential rollout of (5) for the same initial state x_0 and the same randomness $(\xi_t)_{t=1}^T$.*

Proof. If $r(\hat{x}_{1:T}) = 0$, then Proposition 1.2 implies $\hat{x}_{1:T}$ satisfies the recursion (5). That recursion uniquely determines the sequential rollout. ■

Even though the method is solving a problem in $\mathbb{R}^{Td}$, its behavior can be very tangible. Figure 7 shows a one-dimensional ULA example in which we fix the noise $\xi_{1:T}$, start from a crude initial guess, and watch the Gauss–Newton iterates rapidly converge to the sequential rollout.

1.7 When do parallel-in-time trajectory solvers work well?

The parallel-in-time viewpoint gives a family of methods. We fix the randomness and pose trajectory computation as either the rootfinding problem $r(x_{1:T}) = 0$ or the nonlinear least-squares problem

$$\mathcal{L}(x_{1:T}) = \frac{1}{2}\|r(x_{1:T})\|_2^2.$$

By Proposition 1.5, this objective has a unique global minimizer and a unique stationary point (the exact sequential sample path for that fixed randomness). So the global difficulty is not multiple basins, but whether a particular iterative method converges quickly and stably. What can go wrong is divergence, oscillation, or very slow progress.

Stability and contractivity matter in two related ways: they control how errors propagate along the rollout, and they control whether a particular iterative scheme has a large basin of attraction.

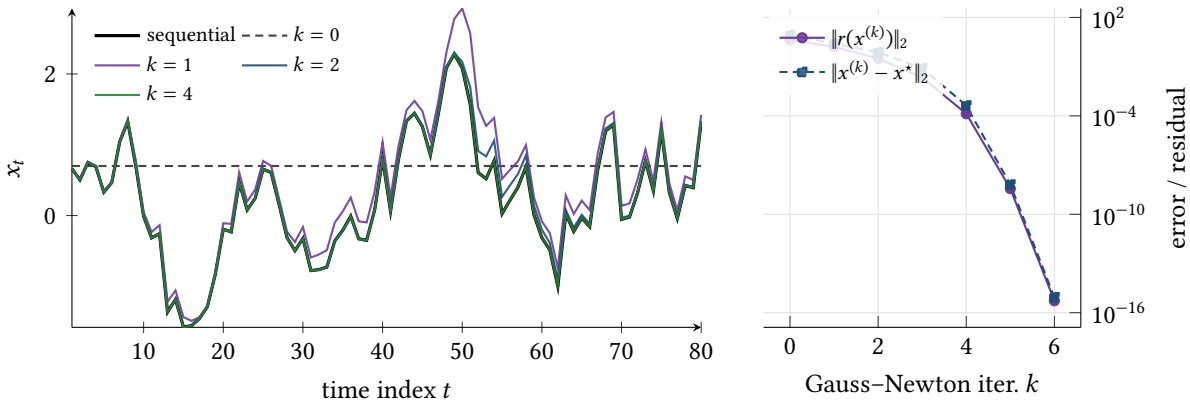

Figure 7: Empirical behavior of the parallel Gauss–Newton rollout on a one-dimensional ULA example with fixed noise. Starting from a crude initial guess, the Gauss–Newton iterates $x_{1:T}^{(k)}$ quickly match the sequential rollout, and the residual norm $\|r(x_{1:T}^{(k)})\|_2$ drops rapidly.

Proposition 1.21 (From residual tolerance to path accuracy)**.** *Fix x_0 and maps $f_{1:T}$. Let $x_{1:T}^\star$ denote the exact rollout and let $\hat{x}_{1:T}$ be any candidate trajectory. Define the residual blocks $\delta_t := r_t(\hat{x}_{1:T})$. Assume each f_t is L_t-Lipschitz on a set containing both trajectories. Then for every t,*

$$\|\hat{x}_t - x_t^\star\|_2 \le \sum_{s=1}^{t} \left(\prod_{j=s+1}^{t} L_j \right) \|\delta_s\|_2,$$

with the convention that an empty product is 1. In particular, when the products $\prod_{j=s+1}^{t} L_j$ stay moderate (a stable/contracting regime), a small residual implies a small path error, whereas in an expanding regime one may need a much tighter residual tolerance to recover the same path accuracy.

Proof. The residual definition says $\hat{x}_1 = f_1(x_0) + \delta_1$ and $\hat{x}_t = f_t(\hat{x}_{t-1}) + \delta_t$ for $t \ge 2$. Subtracting the exact rollout $x_t^\star = f_t(x_{t-1}^\star)$ gives

$$\hat{x}_1 - x_1^\star = \delta_1, \qquad \hat{x}_t - x_t^\star = f_t(\hat{x}_{t-1}) - f_t(x_{t-1}^\star) + \delta_t.$$

Taking norms and using Lipschitzness yields

$$\|\hat{x}_t - x_t^\star\|_2 \le L_t \|\hat{x}_{t-1} - x_{t-1}^\star\|_2 + \|\delta_t\|_2.$$

Unrolling this recursion gives the stated bound. ■

The bounds in Propositions 1.21 and 1.26 show that the same local stability factors govern both how accurately a small-residual trajectory matches the true rollout and how sensitive the Gauss–Newton linear solve is to residuals early in time. Figure 8 gives a visual complement by plotting two-dimensional slices of the high-dimensional merit function $\mathcal{L}(x_{1:T}) = \frac{1}{2}\|r(x_{1:T})\|_2^2$ near the true path. Concretely, we visualize the surface

$$\mathcal{L}\left(x_{1:T}^\star + \alpha v_{\min} + \beta v_{\max}\right),$$

where $v_{\min}$ and $v_{\max}$ are the flattest/steepest directions of the quadratic model $J^\top J$ at $x^\star$. In a stable regime, the basin looks comparatively round; in an expanding regime, it becomes long and thin, reflecting ill-conditioning of the trajectory equations.

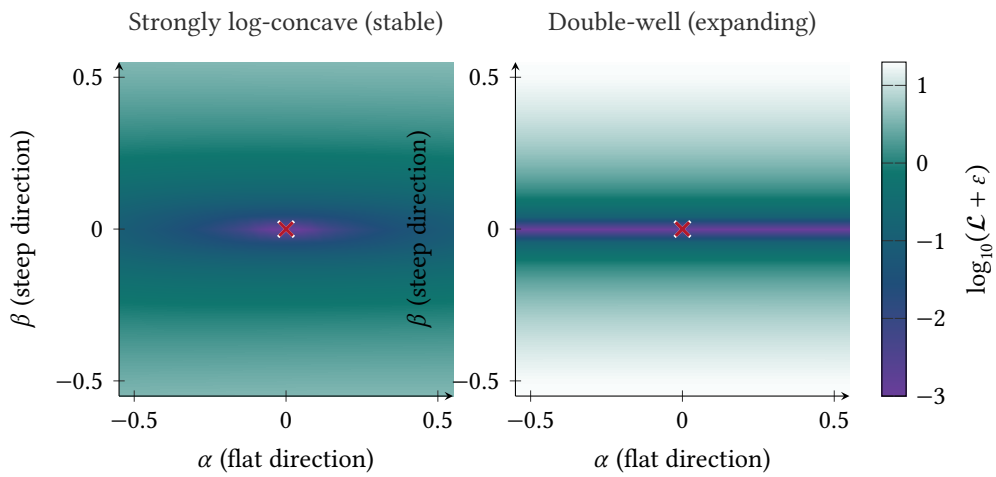

Figure 8: Two-dimensional slices of the trajectory-space merit function near the exact fixed-noise trajectory. The red X marks the exact path $(\alpha, \beta) = (0, 0)$. Stable dynamics yield a comparatively well-conditioned basin (left), while local expansion along the path yields a thin, ill-conditioned basin (right).

The same stability factors that appear in Propositions 1.21 and 1.26 also control the practical difficulty of the trajectory solve: stable dynamics lead to well-conditioned parallel-in-time updates, while expanding dynamics amplify errors and can shrink the basin of attraction of iterative methods. For ULA, strong log-concavity gives an especially clean globally contractive regime.

Proposition 1.22 (A globally contractive regime for the ULA step)**.** *Assume U is twice differentiable and there exist constants $0 < m \le L$ such that*

$$mI_d \preceq \nabla^2 U(x) \preceq LI_d \qquad \text{for all } x \in \mathbb{R}^d.$$

Fix a noise realization ξ_t and consider the corresponding ULA transition map

$$f_t(x) = x - \epsilon \nabla U(x) + \sqrt{2\epsilon}\, \xi_t.$$

If $0 < \epsilon < 2/L$, then for all $x, y \in \mathbb{R}^d$,

$$\|f_t(x) - f_t(y)\|_2 \le \rho \|x - y\|_2, \qquad \rho := \max\{|1 - \epsilon m|,\ |1 - \epsilon L|\} < 1.$$

Consequently, two rollouts with the same noise satisfy

$$\|x_t - x_t'\|_2 \le \rho^t \|x_0 - x_0'\|_2,$$

and the stability factors in Propositions 1.21 and 1.26 decay geometrically.
Moreover, equip trajectories with the max norm $\|x_{1:T}\|_\infty := \max_{t\in\{1,\dots,T\}} \|x_t\|_2$. Then the trajectory map $\mathcal{T}$ from Section 1.1 is a contraction: for all $x_{1:T}, y_{1:T} \in (\mathbb{R}^d)^T$,

$$\|\mathcal{T}(x_{1:T}) - \mathcal{T}(y_{1:T})\|_\infty \le \rho \|x_{1:T} - y_{1:T}\|_\infty.$$

In particular, the fixed-point iteration $x_{1:T}^{(k+1)} = \mathcal{T}(x_{1:T}^{(k)})$ converges from any initialization at rate ρ.

Proof. For fixed ξ_t, the Jacobian of f_t is $I_d - \epsilon\nabla^2 U(x)$. The Hessian bounds imply all eigenvalues of $\nabla^2 U(x)$ lie in $[m, L]$, hence all eigenvalues of $I_d - \epsilon\nabla^2 U(x)$ lie in $[1-\epsilon L,\ 1-\epsilon m]$. Therefore,

$$\|I_d - \epsilon\nabla^2 U(x)\|_{\mathrm{op}} = \max_{\lambda\in[m,L]} |1-\epsilon\lambda| = \max\{|1-\epsilon m|, |1-\epsilon L|\} = \rho.$$

Using the mean value theorem and the uniform bound on the Jacobian gives $\|f_t(x) - f_t(y)\|_2 \le \rho\|x-y\|_2$. Iterating this Lipschitz bound along the recursion yields $\|x_t - x_t'\|_2 \le \rho^t\|x_0 - x_0'\|_2$.

For the trajectory map $\mathcal{T}$, note that $(\mathcal{T}(x))_1 = f_1(x_0)$ does not depend on $x_{1:T}$, so the difference at $t=1$ is 0. For $t \ge 2$,

$$\|(\mathcal{T}(x))_t - (\mathcal{T}(y))_t\|_2 = \|f_t(x_{t-1}) - f_t(y_{t-1})\|_2 \le \rho\|x_{t-1} - y_{t-1}\|_2 \le \rho\|x_{1:T} - y_{1:T}\|_\infty.$$

Taking the maximum over t gives $\|\mathcal{T}(x) - \mathcal{T}(y)\|_\infty \le \rho\|x-y\|_\infty$. If $x^\star$ is the fixed point, then

$$\|x^{(k+1)} - x^\star\|_\infty = \|\mathcal{T}(x^{(k)}) - \mathcal{T}(x^\star)\|_\infty \le \rho\|x^{(k)} - x^\star\|_\infty,$$

so $\|x^{(k)} - x^\star\|_\infty \le \rho^k\|x^{(0)} - x^\star\|_\infty$. The remaining statements follow by substituting $L_j \le \rho$ (or $\alpha_j \le \rho$) into Propositions 1.21 and 1.26. ■

There are multiple equivalent fixed-point formulations for the same fixed-noise trajectory, and contractivity can depend on which one we iterate. The contraction in Proposition 1.22 is for the original rollout map $\mathcal{T}$. By contrast, the scan-friendly cumulative-sum Picard map for ULA in (10) has a sufficient contraction condition that scales with the horizon length (roughly ϵLT in Proposition 1.13), which is why windowing can still be useful for long chains even when the local dynamics are stable.

The next exercise is exactly the same conditioning calculation as for gradient descent, now applied to the ULA transition map.

Exercise 1.23 (Strong convexity and step-size: checking contractivity)**.** Assume U is twice differentiable and $mI_d \preceq \nabla^2 U(x) \preceq LI_d$ for all $x \in \mathbb{R}^d$. Fix $0 < \epsilon < 2/L$ and define $f(x) = x - \epsilon\nabla U(x) + c$ for an arbitrary constant vector $c \in \mathbb{R}^d$.

1. Show that f is ρ-Lipschitz in the Euclidean norm with $\rho = \max\{|1-\epsilon m|, |1-\epsilon L|\}$.
2. For the choice $\epsilon = 1/L$, what is ρ? What happens as $m/L \to 0$?
3. In one sentence, interpret this in terms of the "easy" (contracting) versus "hard" (near-expanding) regimes for parallel-in-time trajectory solvers.

Solution

1. The Jacobian is $Df(x) = I_d - \epsilon\nabla^2 U(x)$. Since the Hessian eigenvalues lie in $[m, L]$, the eigenvalues of $Df(x)$ lie in $[1-\epsilon L,\ 1-\epsilon m]$, so
$$\|Df(x)\|_{\mathrm{op}} = \max_{\lambda\in[m,L]} |1-\epsilon\lambda| = \max\{|1-\epsilon m|, |1-\epsilon L|\} = \rho.$$
The mean value theorem then yields $\|f(x) - f(y)\|_2 \le \rho\|x-y\|_2$ for all x, y.
2. If $\epsilon = 1/L$, then $\rho = \max\{1 - m/L,\ |0|\} = 1 - m/L$. As $m/L \to 0$, we have $\rho \to 1$, meaning the step becomes only weakly contractive (nearly nonexpansive) and perturbations decay very slowly.
3. When the dynamics are strongly contractive (small ρ), errors do not amplify across time and both Picard and Gauss–Newton iterations tend to have large basins and behave robustly; when the map

is near-expansive ($\rho \approx 1$) or expansive, errors can propagate and the trajectory solve can require damping/windows and many more iterations.

At a high level, the wall-clock speed comes from two facts. For both Picard and Gauss–Newton, the expensive local quantities are computed for all t in parallel, and the remaining coupling across time reduces to a scan with $\mathcal{O}(\log T)$ depth. Picard updates are cheap but converge only linearly, while Gauss–Newton updates cost more per iteration but can converge in very few steps when the chain is close to affine along the path.

1.7.1 Picard: contraction, damping, and windows

Picard iteration is the most direct fixed-point method applied to sampling: iterate the trajectory map until it stabilizes. For ULA we wrote this explicitly in (10).

The core requirement is contraction. Proposition 1.13 gives a simple sufficient condition for global convergence on a horizon of length T, and it also explains the main practical lever: reduce the effective horizon. If ϵLT is not small, global Picard can be unstable, but the same dynamics may be stable on a shorter window of length p, which is why sliding-window Picard is natural.

Damping plays the same role as in optimization: the relaxation $x^{(k+1)} \leftarrow (1-\omega)x^{(k)} + \omega\tilde{x}^{(k+1)}$ can turn a borderline-stable iteration into a convergent one, at the cost of more iterations.

1.7.2 First-order optimization on $\mathcal{L}$: a clean rate in the quadratic case

Picard is not the only first-order-style approach. We can also apply standard optimization methods directly to the trajectory objective $\mathcal{L}(x_{1:T}) = \frac{1}{2}\|r(x_{1:T})\|_2^2$. In general $\mathcal{L}$ is nonconvex (as discussed in Section 1.5), but in the quadratic ULA warm-up (Section 1.3) it becomes a strongly convex quadratic, so gradient descent admits a transparent convergence-rate calculation.

Proposition 1.24 (Gradient descent on the trajectory objective in the affine case)**.** *Assume the residual is affine, $r(x) = Jx - c$, where $J \in \mathbb{R}^{Td\times Td}$ is invertible and $c \in \mathbb{R}^{Td}$. Define $\mathcal{L}(x) = \frac{1}{2}\|Jx - c\|_2^2$ and let $x^\star = J^{-1}c$. Let $H := \nabla^2\mathcal{L} = J^\top J$ and define $\mu := \lambda_{\min}(H) > 0$ and $M := \lambda_{\max}(H)$. Then gradient descent with step size $\alpha = 1/M$,*

$$x^{(k+1)} = x^{(k)} - \alpha\nabla\mathcal{L}(x^{(k)}),$$

satisfies, for all $k \geq 0$,

$$\mathcal{L}(x^{(k)}) \leq \left(1 - \frac{\mu}{M}\right)^k \mathcal{L}(x^{(0)}).$$

In particular, to reduce the objective by a factor $\delta \in (0,1)$ it suffices to take

$$k \;\geq\; \frac{M}{\mu}\log\left(\frac{1}{\delta}\right) \;=\; \kappa(H)\log\left(\frac{1}{\delta}\right),$$

where $\kappa(H) = M/\mu$ is the condition number of the quadratic model.

Proof. Since $r(x) = Jx - c$ is affine,

$$\nabla\mathcal{L}(x) = J^\top(Jx - c) = H(x - x^\star),$$

so $\mathcal{L}(x) = \frac{1}{2}(x - x^\star)^\top H(x - x^\star)$ is a μ-strongly convex, M-smooth quadratic. The standard gradient-descent

rate for strongly convex, smooth functions gives $\mathcal{L}(x^{(k)}) \le (1-\mu/M)^k \mathcal{L}(x^{(0)})$. Solving $(1-\mu/M)^k \le \delta$ yields the iteration bound. ■

In the quadratic ULA setting, Proposition 1.18 shows that Gauss–Newton solves the same affine trajectory equations in *one* iteration (just as Newton solves quadratic objectives in one step). Proposition 1.24 is therefore the classic "gradient descent versus Newton" tradeoff, but now in trajectory space: first-order methods can require $\mathcal{O}(\kappa)$ iterations, while the scan-based linear solve in Gauss–Newton can remove that dependence at the cost of more per-iteration work.

From a parallelism perspective, when $r(x) = Jx - c$ is affine and J is block bidiagonal (as in our rollout residual), one gradient-descent iteration is highly parallel across time: each residual block depends only on (x_{t-1}, x_t), and each gradient block depends only on neighboring residuals. So with sufficient parallelism, one GD step has $\mathcal{O}(1)$ depth after computing any needed per-time Jacobians. By contrast, the scan-based primitives in Picard and Gauss–Newton have $\mathcal{O}(\log T)$ depth per iteration. The wall-clock advantage therefore depends on iteration count: sequential rollout takes $\mathcal{O}(T)$ time, while a K-iteration scan-based trajectory solver costs roughly $\mathcal{O}(K \log T)$ parallel rounds.

1.7.3 Second-order (Gauss–Newton): mild nonlinearity and good local models

Gauss–Newton is the natural "Newton-like" method for the trajectory objective $\mathcal{L} = \frac{1}{2}\|r\|_2^2$: it repeatedly builds a local linear model for the residual r and uses that model to propose an update of the entire trajectory. The mental model is Proposition 1.18: if the transitions are affine, one Gauss–Newton step solves the trajectory equations from any initialization. For nonlinear transitions, Gauss–Newton repeatedly linearizes each step around the current trajectory guess, so it works best when the chain is *close to affine* on the region explored by the path.

Proposition 1.25 (Local quadratic convergence of undamped Gauss–Newton in a basin)**.** *Let $r : \mathbb{R}^{Td} \to \mathbb{R}^{Td}$ be continuously differentiable and suppose $x^\star \in \mathbb{R}^{Td}$ satisfies $r(x^\star) = 0$. Assume there is a radius $\rho > 0$ such that on the closed ball $B(x^\star, \rho)$ the Jacobian $J(x) := Dr(x)$ is L_J-Lipschitz and invertible with*

$$\|J(x)^{-1}\|_{\mathrm{op}} \le M.$$

Consider the undamped Gauss–Newton iteration

$$x^{(k+1)} = x^{(k)} - J(x^{(k)})^{-1} r(x^{(k)}).$$

If $\|x^{(0)} - x^\star\|_2 \le \min\{\rho,\ 1/(L_J M)\}$, then the iterates are well-defined, remain in $B(x^\star, \rho)$, and satisfy the quadratic bound

$$\|x^{(k+1)} - x^\star\|_2 \le \frac{L_J M}{2} \|x^{(k)} - x^\star\|_2^2 \qquad \text{for all } k \ge 0.$$

In particular, once the iteration enters this basin, convergence is quadratic.

Proof. Fix k and write $e := x^{(k)} - x^\star$. By the fundamental theorem of calculus,

$$r(x^{(k)}) - r(x^\star) = \int_0^1 J(x^\star + \tau e)\, e \, d\tau.$$

Subtracting $J(x^{(k)})e$ and using Lipschitzness of J gives

$$\|r(x^{(k)}) - r(x^\star) - J(x^{(k)})e\|_2 \le \int_0^1 \|J(x^\star + \tau e) - J(x^{(k)})\|_{\mathrm{op}} \|e\|_2 \, d\tau \le \int_0^1 L_J (1-\tau) \|e\|_2^2 \, d\tau = \frac{L_J}{2}\|e\|_2^2.$$

Since $r(x^\star) = 0$, the Gauss–Newton update gives

$$x^{(k+1)} - x^\star = x^{(k)} - x^\star - J(x^{(k)})^{-1}\big(r(x^{(k)}) - r(x^\star)\big) = -J(x^{(k)})^{-1}\big(r(x^{(k)}) - r(x^\star) - J(x^{(k)})e\big).$$

Taking norms and using $\|J(x^{(k)})^{-1}\|_{\mathrm{op}} \le M$ yields

$$\|x^{(k+1)} - x^\star\|_2 \le \frac{L_J M}{2}\|x^{(k)} - x^\star\|_2^2.$$

If $\|x^{(0)} - x^\star\|_2 \le \min\{\rho, 1/(L_J M)\}$, then $x^{(0)} \in B(x^\star, \rho)$ and the first Newton step is well-defined. The bound implies $\|x^{(1)} - x^\star\|_2 \le 1/(2L_J M) \le \rho$. Inductively, if $x^{(k)} \in B(x^\star, \rho)$ and $\|x^{(k)} - x^\star\|_2 \le 1/(L_J M)$, then the next step is well-defined and again satisfies $\|x^{(k+1)} - x^\star\|_2 \le 1/(2L_J M) \le \rho$, so all iterates remain in the ball where the assumptions hold. ■

For ULA, a few recurring "good" conditions are worth keeping in mind. When U is twice differentiable, the step Jacobians exist and are easy to write down; Exercise 1.15 gives $A_t^{(k)} = I_d - \epsilon\nabla^2 U(x_{t-1}^{(k)})$. If U is close to quadratic on the region explored by the trajectory, then the linearizations are accurate and Gauss–Newton converges quickly. A practical proxy is that smaller ϵ makes each step closer to "identity plus a small perturbation," which typically improves the stability of the linear solve. Because Gauss–Newton is local, a reasonable initial trajectory also matters. Starting from a crude guess such as $x_t^{(0)} \equiv x_0$ can work, as in Figure 7, but warm starts usually reduce the iteration count. When a full step overshoots, a line search on $\mathcal{L}(x_{1:T}) = \frac{1}{2}\|r(x_{1:T})\|_2^2$ can enforce progress and stabilize convergence.

Stability still matters for Gauss–Newton: the linear solve is a forward recursion, so large "expansion factors" can amplify early-time errors (and make steps sensitive); see Proposition 1.26.

Proposition 1.26 (Amplification in the Gauss–Newton linear solve)**.** *Fix an iteration k and consider the recursion (11). Assume $\|A_t^{(k)}\|_{\mathrm{op}} \le \alpha_t$ for $t = 2, \dots, T$. Then the Gauss–Newton increment satisfies, for each t,*

$$\|\Delta x_t^{(k)}\|_2 \le \sum_{s=1}^{t}\left(\prod_{j=s+1}^{t}\alpha_j\right)\|r_s^{(k)}\|_2,$$

with the convention that an empty product is 1. In particular, if products of the form $\prod_{j=s+1}^{t}\alpha_j$ are large, small residuals early in time can induce large changes later in the path, making the step sensitive and potentially requiring damping.

Proof. For $t = 1$, $\Delta x_1^{(k)} = -r_1^{(k)}$. For $t \ge 2$, unrolling (11) gives

$$\Delta x_t^{(k)} = -r_t^{(k)} - A_t^{(k)} r_{t-1}^{(k)} - \cdots - \left(A_t^{(k)} A_{t-1}^{(k)} \cdots A_2^{(k)}\right) r_1^{(k)}.$$

Taking norms and using submultiplicativity, $\|A_j^{(k)}\|_{\mathrm{op}} \le \alpha_j$, yields the stated bound. ■

1.7.4 Picard vs Gauss–Newton: a systematic comparison

Both methods compute the *same* fixed-noise trajectory when they converge. The difference is how they trade off per-iteration work against iteration count. The table below summarizes the main points in the ULA setting.

	Picard iteration	**Gauss–Newton**
What is iterated?	Picard fixed-point iteration on the trajectory equations (for ULA, (10))	Newton/Gauss–Newton updates for solving $r(x_{1:T}) = 0$ (equivalently, minimizing $\mathcal{L}(x_{1:T}) = \frac{1}{2}\|r(x_{1:T})\|_2^2$)
Per-iter ingredients	Drift / increment evaluations (e.g. ∇U)	Drift + Jacobians (for ULA, $\nabla^2 U$)
Parallel primitive	Prefix-sum scan to integrate increments (ULA)	Scan-based solve of a block bidiagonal linear system
Convergence type	Linear; needs contraction (or damping / windows)	Fast local convergence when the model is accurate; may need damping
Main failure mode	Divergence or very slow progress when the map is not contracting over the horizon	Overshoot / poor linear model far from the solution; sensitivity when expansion factors are large
Typical fixes	Smaller windows p, damping ω, smaller step size ϵ	Warm starts, line search/damping, smaller step size ϵ
When to prefer	Derivatives unavailable or drift evaluations are very cheap and dynamics are stable	Derivatives available and transitions are close to affine along the path (few iterations)

Finally, this comparison extends beyond ULA. For a general differentiable transition f_t, Picard iteration is still the generic fixed-point iteration $x^{(k+1)} = \mathcal{T}(x^{(k)})$ (with all $f_t(x_{t-1}^{(k)})$ computable in parallel), and Gauss–Newton still uses the Jacobian structure of the residual to build scan-friendly linear solves. When the transition is not differentiable (e.g. accept/reject), the fixed-point viewpoint still provides structure, but Gauss–Newton-style updates require additional approximations.

1.8 Extensions beyond ULA

The setup in Section 1.1 was to fix the randomness and view the resulting chain as the solution of a deterministic rollout recursion (2). That viewpoint is not special to ULA. We briefly sketch a few other cases without changing the core story.

The same caution about convexity applies here too: once the randomness is fixed, we can still write a trajectory residual and a least-squares objective in trajectory space, but for more complex transitions those objectives are typically nonconvex and may be nonsmooth, for example because of accept/reject steps. The point of the rollout viewpoint is structure, not a promise that sampling has become a convex optimization problem.

Definition 1.27 (Reparameterized transition map)**.** Fix a distribution μ on an auxiliary randomness space and a measurable map

$$F : \mathbb{R}^d \times \Xi \to \mathbb{R}^d.$$

Given i.i.d. random variables $\xi_1, \dots, \xi_T \sim \mu$ and an initial state x_0, define $f_t(\cdot) := F(\cdot, \xi_t)$. Then the chain is the unique sequence satisfying the rollout recursion (5).

Examples follow the same pattern. Reparameterized Gibbs sampling writes each conditional draw as a deterministic function of a fresh uniform or Gaussian variable, for example through inverse-CDF sampling. SGLD replaces ∇U in ULA by a stochastic gradient estimator, but conditional on the minibatch randomness and Gaussian noise the update is still a deterministic recursion. Metropolis-corrected chains include the uniform random variable used for accept/reject; the update is still a deterministic map of the current state and the random inputs, but it is no longer differentiable.

1.8.1 Reparameterizing a Gibbs update via inverse CDF

Exercise 1.28 (Inverse-CDF sampling)**.** Consider a one-dimensional conditional distribution with CDF $G(\cdot \mid \text{rest})$. Define the generalized inverse by

$$G^{-1}(u \mid \text{rest}) := \inf\{x \in \mathbb{R} : G(x \mid \text{rest}) \geq u\}.$$

Show that if $U \sim \text{Unif}(0,1)$ and $X = G^{-1}(U \mid \text{rest})$, then X has the desired conditional distribution. Explain briefly how this turns a Gibbs sweep into a rollout of the form (5).

Solution

Let $G(\cdot \mid \text{rest})$ be a conditional CDF and define the generalized inverse

$$G^{-1}(u \mid \text{rest}) := \inf\{x \in \mathbb{R} : G(x \mid \text{rest}) \geq u\}.$$

Set $X := G^{-1}(U \mid \text{rest})$ with $U \sim \text{Unif}(0,1)$ independent of rest. For any x, the defining property of the generalized inverse gives

$$\{X \leq x\} \iff \{G^{-1}(U \mid \text{rest}) \leq x\} \iff \{U \leq G(x \mid \text{rest})\}.$$

Taking conditional probabilities given rest yields

$$\mathbb{P}(X \leq x \mid \text{rest}) = \mathbb{P}(U \leq G(x \mid \text{rest}) \mid \text{rest}) = G(x \mid \text{rest}),$$

since U is uniform on $(0,1)$. Therefore X has conditional CDF $G(\cdot \mid \text{rest})$ as desired.
In a Gibbs sampler, each coordinate (or block) update draws from a conditional distribution. Replacing each draw by an inverse-CDF transform of an independent uniform random variable makes the update a deterministic function of the current state and the new randomness. Composing the coordinate/block updates within one sweep yields a transition map F, so the chain can be written as $x_t = F(x_{t-1}, \xi_t)$ and hence as a rollout (5).

1.8.2 Metropolis corrections (RWMH and MALA)

Random-walk Metropolis–Hastings (RWMH) and the Metropolis-adjusted Langevin algorithm (MALA) add an accept/reject correction. For example, in RWMH with a symmetric Gaussian proposal,

$$\tilde{x}_t = x_{t-1} + \delta\xi_t, \qquad \xi_t \sim \mathcal{N}(0, I_d), \qquad \alpha(x_{t-1}, \tilde{x}_t) = \min\left\{1, \frac{\pi(\tilde{x}_t)}{\pi(x_{t-1})}\right\}.$$

In MALA, the proposal is the ULA step (4), but one still accepts/rejects using the Metropolis–Hastings ratio (with the Gaussian proposal densities).

Given a proposal $\tilde{x}_t$ and an acceptance probability $\alpha(x_{t-1}, \tilde{x}_t) \in [0,1]$, draw $u_t \sim \text{Unif}(0,1)$ and set

$$x_t = \begin{cases} \tilde{x}_t, & u_t \leq \alpha(x_{t-1}, \tilde{x}_t), \\ x_{t-1}, & \text{otherwise.} \end{cases} \tag{14}$$

This is still a deterministic map of (x_{t-1}, ξ_t, u_t), and therefore still fits Definition 1.27. The complication is differentiability: the accept/reject decision is a hard threshold, so the Jacobians needed for Gauss–Newton updates are not well defined everywhere.

Exercise 1.29 (Metropolis gating and differentiability)**.** Consider the accept/reject step (14).

1. Show that it can be rewritten in the "gating" form $x_t = g_t \tilde{x}_t + (1 - g_t)x_{t-1}$, where $g_t \in \{0, 1\}$ depends on (x_{t-1}, ξ_t, u_t).
2. Explain why x_t is generally not differentiable as a function of x_{t-1}.
3. Propose one smooth surrogate $\tilde{g}_t \in (0, 1)$ that could be used *only* to compute an approximate Jacobian, while keeping the forward update (14) unchanged.

Solution

1. Define $g_t := \mathbf{1}\{u_t \leq \alpha(x_{t-1}, \tilde{x}_t)\} \in \{0, 1\}$. Then (14) is equivalent to

$$x_t = g_t \tilde{x}_t + (1 - g_t)x_{t-1}.$$

   Indeed, if $g_t = 1$ (accept) this gives $x_t = \tilde{x}_t$, and if $g_t = 0$ (reject) it gives $x_t = x_{t-1}$.

2. The map $x_{t-1} \mapsto g_t$ contains an indicator of an inequality. Unless $\alpha(x_{t-1}, \tilde{x}_t)$ is constant in x_{t-1} (a degenerate case), the set where g_t flips from 0 to 1 is a decision boundary, across which g_t jumps discontinuously. Since x_t depends on g_t, the overall map $x_{t-1} \mapsto x_t$ is typically discontinuous in its derivative (and often discontinuous in the function itself when $\tilde{x}_t$ depends on x_{t-1}), so it is not differentiable everywhere.

3. One option is a logistic (soft) gate with temperature $\tau > 0$:

$$\tilde{g}_t := \sigma\left(\frac{\log \alpha(x_{t-1}, \tilde{x}_t) - \log u_t}{\tau}\right) \in (0, 1),$$

   where $\sigma(s) = 1/(1 + e^{-s})$. As $\tau \downarrow 0$, $\tilde{g}_t$ becomes a sharper approximation to the hard gate $\mathbf{1}\{\log \alpha - \log u_t \geq 0\}$. In an inexact Gauss–Newton implementation one would keep the forward update using the true hard gate g_t, but use $\tilde{g}_t$ (or its Jacobian contribution) when forming an approximate Jacobian for the Gauss–Newton step.

1.9 Burn-in and stationarity: when does the trajectory solve get easier?

So far, we have fixed the random inputs $\xi_{1:T}$ and asked how to recover the corresponding length-T trajectory in parallel. Burn-in is usually introduced as a statistical issue: early states may be far from the target distribution π. For parallel-in-time methods there is also a computational issue: the difficulty of solving the fixed-noise trajectory equations depends on how perturbations propagate along the rollout.

The next proposition isolates the basic mechanism. It is simply a stability bound for the deterministic dynamics obtained after conditioning on the random inputs.

Proposition 1.30 (Away from stationarity: local stability can make the parallel solve easier or harder)**.** *Fix a realization of the random inputs $\xi_{1:T}$ used by the sampler and write the resulting deterministic transitions as maps f_t. Let $x_{1:T}$ and $x'_{1:T}$ be the rollouts from two initial states x_0 and x'_0 using the* same *realization $\xi_{1:T}$. Assume each f_t is L_t-Lipschitz on a set containing both trajectories. Then, for every t,*

$$\|x_t - x'_t\|_2 \le \left(\prod_{s=1}^{t} L_s\right) \|x_0 - x'_0\|_2.$$

In particular, if these products stay moderate (a stable/contracting regime), early-time errors are not amplified across the chain, while if they grow quickly (an expanding regime), small early-time errors can blow up at later times.

Proof. Let $t \ge 1$. Using the rollout recursion and the Lipschitz property,

$$\|x_t - x'_t\|_2 = \|f_t(x_{t-1}) - f_t(x'_{t-1})\|_2 \le L_t \, \|x_{t-1} - x'_{t-1}\|_2.$$

Iterating this inequality from 1 to t yields the stated bound. ■

This perspective connects directly to our two trajectory solvers. Picard is the fixed-point method: in a contracting regime it converges from any initialization, but when the map is not contractive over the full horizon it can diverge or move slowly, which is why damping and windowing are useful. Gauss–Newton is the second-order method: its linearized solve is always well-defined because the residual Jacobian is block lower triangular with identity blocks on the diagonal (Proposition 1.3), but in an expanding regime Proposition 1.26 shows that early residual errors can be amplified in the forward recursion, so damping, line search, and good warm starts matter most.

For ULA with fixed noise and twice differentiable U, the transition maps are

$$f_t(x) = x - \epsilon \nabla U(x) + \sqrt{2\epsilon}\, \xi_t, \qquad \frac{\partial f_t}{\partial x}(x) = I_d - \epsilon \nabla^2 U(x).$$

Thus curvature of U (and the step size ϵ) controls whether f_t is locally contracting or expanding along the path. In strongly log-concave targets with globally bounded curvature, Proposition 1.22 gives a clean globally contractive regime, so the stability factors in Propositions 1.21 and 1.26 stay controlled. Away from stationarity, the chain can pass through high-curvature (or negatively curved) regions, increasing local expansion factors and making windowing/damping more important.

More broadly, this fixed-randomness trajectory optimization viewpoint and its stability/conditioning issues are not special to MCMC. They also appear when parallelizing other sequential computations, including nonlinear state-space models and bridge-based diffusion sampling; see, e.g., Lim et al. (2024, §3), Gonzalez et al. (2024, §1), and Shih et al. (2023, §3).

2 Diagnosing when parallel-in-time MCMC is worthwhile

The final diagnosis is different from the nonconvex examples in the Optimization unit and from the expectation–maximization subsection in the Augmented-state optimization unit. For fixed randomness, the trajectory problem has one exact solution, so the main question is not which optimum exists but whether the iterative solver can reach that rollout quickly and stably. The useful checks are therefore numerical and geometric. One first asks whether the fixed-noise residual problem has the exact rollout as its unique stationary point, so that the issue is conditioning rather than competing minima. One then asks whether the products of local Lipschitz factors stay moderate along the horizon of interest, whether the rollout is contractive enough for Picard on the full horizon or instead requires damping and windows, whether the trajectory is smooth enough that Gauss–Newton has a useful linear model, and whether the solve passes through an expanding burn-in regime or a near-stationary regime where early errors are not strongly amplified.

When those checks look favorable, scan-based Picard or Gauss–Newton updates can recover long rollouts substantially faster than one step at a time. When they fail, the right repair is usually windowing, damping, or a better local surrogate rather than a different sampling target. Carried back to the main notes, the lesson is that parallel-in-time sampling is not a new sampling principle. It is a numerical method for solving one realized trajectory, and its success is governed by the same conditioning and local-model questions that already control iterative methods elsewhere in the course.

References

Xavier Gonzalez. *Parallelizing "Inherently Sequential" Processes: Methods for Efficient Evaluation of Nonlinear Recurrent Models.* PhD thesis, Stanford University, February 2026.

Xavier Gonzalez, Andrew Warrington, Jimmy T. H. Smith, and Scott W. Linderman. Towards scalable and stable parallelization of nonlinear RNNs, 2024.

Yi Heng Lim, Qi Zhu, Joshua Selfridge, and Muhammad Firmansyah Kasim. Parallelizing non-linear sequential models over the sequence length, 2024.

Andy Shih, Suneel Belkhale, Stefano Ermon, Dorsa Sadigh, and Nima Anari. Parallel sampling of diffusion models, 2023.

David M. Zoltowski, Skyler Wu, Xavier Gonzalez, Leo Kozachkov, and Scott W. Linderman. Parallelizing MCMC across the sequence length, 2025.